

ADC核心概念详细解析

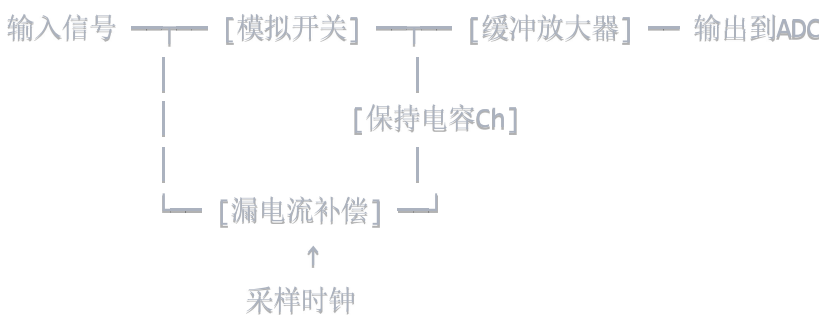
1. 采样保持 (Sample and Hold)

基本原理

采样保持是ADC的第一步，解决了一个根本矛盾：**模拟信号是连续变化的，但ADC需要时间来完成转换。**

物理实现

基本电路结构



工作时序

1. 采样相(Track Phase):

- 开关闭合，电容跟踪输入信号
- 持续时间：几纳秒到几十纳秒
- 电容充电到输入电压值

2. 保持相(Hold Phase):

- 开关断开，电容保持电压不变
- 持续时间：ADC转换所需的全部时间
- 电压必须保持稳定，误差小于0.5LSB

关键性能参数

1. 采样时间窗口(Aperture Time)

- 定义：开关从导通到完全断开的時間
- 典型值：1-10ps
- 影响：限制ADC的最高输入频率

2. 采样时间不确定性(Aperture Jitter)

- 定义：采样时刻的随机变化

- 影响：产生额外的噪声，降低SNR
- 计算公式： $SNR_{jitter} = -20\log_{10}(2\pi f_{in} \times t_{jitter})$

3. 保持时间(Hold Time)

- 电容上电压的衰减时间常数
- 主要由漏电流决定： $\tau = C_h \times R_{leakage}$
- 要求：在整个转换期间，电压下降 $< 0.5LSB$

实际挑战

电荷注入效应：

- 开关断开时，沟道电荷被"踢入"保持电容
- 造成电压误差： $\Delta V = Q_{channel} / C_h$
- 解决方案：差分结构、电荷补偿技术

时钟馈通：

- 采样时钟通过寄生电容耦合到信号路径
- 解决方案：虚拟时钟技术、屏蔽设计

应用实例

例：100MHz ADC，输入信号10MHz正弦波

- 采样率：100MS/s
- 采样窗口：~100ps
- 保持时间：~10ns
- 如果时钟抖动1ps，SNR损失约0.5dB

2. 量化与量化噪声

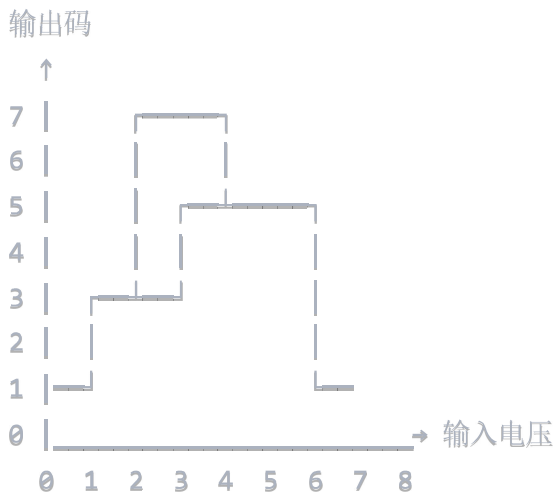
量化过程详解

数学描述

对于N位ADC，满量程电压 V_{FS} ：

- LSB步长**： $LSB = V_{FS} / 2^N$
- 量化函数**： $Q(x) = LSB \times \text{round}(x/LSB)$
- 量化误差**： $e(x) = Q(x) - x$

量化特性曲线



量化噪声的统计特性

理想情况下的假设

1. **均匀分布**：量化误差在 $[-\text{LSB}/2, +\text{LSB}/2]$ 范围内均匀分布
2. **白噪声**：各次采样的量化误差相互独立
3. **与信号无关**：量化噪声与输入信号统计独立

量化噪声功率计算

方差： $\sigma^2_q = \int_{-\text{LSB}/2}^{+\text{LSB}/2} e^2 \times (1/\text{LSB}) \, de = \text{LSB}^2/12$

RMS值： $\sigma_q = \text{LSB}/\sqrt{12} \approx 0.289 \times \text{LSB}$

功率谱密度： $S_q(f) = \text{LSB}^2/(12 \times f_s)$ （白噪声谱）

信噪比计算

对于满量程正弦波输入：

信号功率： $P_s = (V_{FS}/2)^2/2 = V_{FS}^2/8$

噪声功率： $P_n = \text{LSB}^2/12 = (V_{FS}/2^N)^2/12$

$\text{SNR} = 10 \times \log_{10}(P_s/P_n) = 6.02N + 1.76 \text{ dB}$

这就是著名的"每位6dB"定律

非理想因素

1. 大信号量化噪声

当输入信号幅度接近满量程时：

- 量化误差不再均匀分布
- 产生谐波失真
- SNR会偏离理论值

2. 小信号量化噪声

当输入信号很小时：

- 量化噪声与信号相关性增强
- 可能出现"极限环"现象
- 需要添加抖动(Dither)信号

3. 直流偏移的影响

如果ADC有直流偏移：

- 量化点发生偏移
- 影响量化误差的统计特性
- 可能产生音频"咔嗒"声

3. 余差放大 (Residue Amplification)

Pipeline ADC的基本原理

Pipeline ADC将转换过程分成多个级联的阶段，每个阶段完成部分位的转换。

单级结构



详细工作流程

第一步：粗量化

- 用低精度ADC(如3位Flash)进行粗转换
- 得到最高几位的数字码：D_coarse

第二步：余差计算

- 将数字码通过DAC转换回模拟信号：V_DAC
- 计算余差：V_residue = V_in - V_DAC

第三步：余差放大

- 将余差信号放大: $V_{\text{amplified}} = G \times V_{\text{residue}}$
- 增益G通常等于粗ADC的量化步数: $G = 2^{n_{\text{coarse}}}$

数学分析

理想情况

假设第一级为3位转换:

$V_{\text{in}} = 2.7\text{V}$ (满量程8V)
粗转换: $D_{\text{coarse}} = 3$ (二进制011)
DAC输出: $V_{\text{DAC}} = 3\text{V}$
余差: $V_{\text{residue}} = 2.7\text{V} - 3\text{V} = -0.3\text{V}$
放大后: $V_{\text{amplified}} = 8 \times (-0.3\text{V}) = -2.4\text{V}$

增益误差的影响

如果放大器增益不准确:

- **增益过大:** 下一级会饱和, 产生非线性
- **增益过小:** 动态范围没有充分利用, 增加噪声

增益误差容限: 通常要求 $< 0.1\%$

关键技术挑战

1. 运算放大器设计

性能要求:

- **增益精度:** $< 0.1\%$ 误差
- **建立时间:** 必须在半个时钟周期内稳定
- **噪声:** 不能显著恶化总SNR
- **线性度:** 避免引入谐波失真

设计权衡:

速度 ↔ 精度 ↔ 功耗

高速 → 增益带宽积要求高 → 功耗大
高精度 → 需要大增益 → 稳定性难保证
低功耗 → 驱动能力弱 → 建立时间长

2. 电容匹配

在开关电容实现中：

- 采样电容和反馈电容的比值决定增益
- 电容失配直接影响增益精度
- 通常需要激光修调或数字校准

3. 时钟馈通和电荷注入

- 开关操作会引入误差信号
- 在高精度应用中必须仔细处理
- 常用差分结构和虚拟开关技术

数字校准技术

后台校准算法

```
matlab

% 伪代码示例
for each pipeline stage i:
    measure actual gain G_i
    calculate ideal gain G_ideal = 2^n_i
    store correction factor K_i = G_ideal / G_i

during normal operation:
    digital_output = raw_output * K_i
```

4. 多级量化 (Multi-stage Quantization)

设计动机

单级量化的局限性：

- Flash ADC**：比较器数量指数增长 ($2^N - 1$ 个)
- SAR ADC**：转换时间线性增长 (N个时钟周期)
- 功耗**：随精度快速增长

多级量化的优势：

- 降低复杂度**：将N位分解为多个n位 ($n \ll N$)
- 提高速度**：并行或流水线处理
- 优化功耗**：避免大量冗余比较器

不同架构的多级实现

1. Pipeline ADC

时间分级：

时钟周期1: Stage1处理Sample1, 输出MSB
时钟周期2: Stage1处理Sample2, Stage2处理Sample1余差
时钟周期3: Stage1处理Sample3, Stage2处理Sample2, Stage3处理Sample1
...

优点：

- 高吞吐率：每个时钟周期输出一个完整转换结果
- 功耗相对较低
- 容易实现高精度

缺点：

- 延迟时间长：需要等待流水线填满
- 复杂的数字校准

2. 多步Flash ADC

空间分级：

第一步：粗Flash ADC（如4位）→ 确定MSB
第二步：细Flash ADC（如4位）→ 确定LSB
总计：8位精度，只需2×15=30个比较器（vs 255个）

3. 分段式SAR ADC

逻辑分级：

第一段：SAR确定高6位
第二段：SAR确定低6位
总计：12位精度，DAC复杂度降低

性能权衡分析

速度分析

N位转换，分成k级，每级n位 (N = k×n):

Flash: 1个时钟周期

Pipeline: k+1个时钟周期 (考虑延迟)

SAR: N个时钟周期

对于14位ADC:

- 单级Flash: 16383个比较器, 1 cycle
- 7级Pipeline: 7×3=21个比较器, 8 cycles
- 单级SAR: 1个比较器, 14 cycles

精度分析

各级的误差会累积:

总INL ≈ √(INL₁² + (INL₂/G₁)² + (INL₃/G₁G₂)² + ...)

其中G₁是第1级前的总增益

可见后级的精度要求相对较低

5. 码字拼接 (Code Stitching)

Pipeline ADC中的码字对齐

时间延迟问题

由于流水线结构，不同位的数据在不同时刻产生:

Sample 1:

- t=1: B13,B12,B11 (Stage1输出)
- t=2: B10,B9,B8 (Stage2输出)
- t=3: B7,B6,B5 (Stage3输出)
- t=4: B4,B3,B2 (Stage4输出)
- t=5: B1,B0 (Stage5输出)

数字延迟链

verilog

```
// VeriLog伪代码
reg [2:0] stage1_out, stage1_d1, stage1_d2, stage1_d3, stage1_d4;
reg [2:0] stage2_out, stage2_d1, stage2_d2, stage2_d3;
reg [2:0] stage3_out, stage3_d1, stage3_d2;
reg [2:0] stage4_out, stage4_d1;
reg [1:0] stage5_out;

always @(posedge clk) begin
    // 延迟链对齐
    stage1_d1 <= stage1_out;
    stage1_d2 <= stage1_d1;
    stage1_d3 <= stage1_d2;
    stage1_d4 <= stage1_d3;

    // 最终输出
    final_code = {stage1_d4, stage2_d3, stage3_d2, stage4_d1, stage5_out};
end
```

数字误差校正

冗余位技术

为了提高精度，通常引入冗余位：

- 每级不是严格的 n 位，而是 $n+1$ 位
- 允许级间增益有一定误差
- 通过数字算法校正最终结果

校正算法示例

matlab

```
function corrected_code = digital_correction(raw_stages, cal_weights)
% raw_stages: 各级的原始输出
% cal_weights: 校准得到的实际权重

corrected_code = 0;
for i = 1:length(raw_stages)
    corrected_code = corrected_code + raw_stages(i) * cal_weights(i);
end
end
```

6. 权重校准 (Weight Calibration)

校准的必要性

误差来源分析

1. 电容失配 (SAR ADC)

理想: $C_0=C, C_1=2C, C_2=4C, C_3=8C$

实际: 存在随机误差 σ 和系统误差 δ

$$C_0 = C(1 + \delta_0 + \sigma_0)$$

$$C_1 = 2C(1 + \delta_1 + \sigma_1)$$

...

其中 $\sigma_1 \sim N(0, \sigma^2_{\text{match}})$, δ_1 为系统偏差

2. 运放增益误差 (Pipeline ADC)

理想增益: $G = 2^{n_{\text{stage}}}$

实际增益: $G_{\text{actual}} = G(1 + \epsilon_G)$

其中 ϵ_G 包括:

- 有限直流增益误差
- 有限带宽引起的误差
- 非线性误差

3. 电阻失配 (Flash ADC)

参考电压梯: $V_{\text{ref}}/2^N \times k, k=1,2,\dots,2^N-1$

实际阶梯: 存在累积误差, 影响判决门限

校准方法详解

1. 静态校准 (Foreground Calibration)

基本流程:

- 停止正常ADC操作
- 施加精确的校准信号
- 测量各位的实际权重
- 存储校准系数

权重测量算法:

matlab

```
function weights = measure_weights(adc, n_bits)
    weights = zeros(1, n_bits);

    % 测量每一位的权重
    for bit = 1:n_bits
        % 施加使该位翻转的信号
        input_low = generate_signal_for_bit(bit, 0);
        input_high = generate_signal_for_bit(bit, 1);

        % 多次测量求平均
        output_low = mean(adc_convert(input_low, 1000));
        output_high = mean(adc_convert(input_high, 1000));

        % 计算实际权重
        weights(bit) = output_high - output_low;
    end
end
```

2. 后台校准 (Background Calibration)

优势：

- 不中断正常操作
- 能跟踪温度、老化等长期漂移
- 适合高可靠性应用

挑战：

- 需要区分校准信号和输入信号
- 算法复杂度高
- 收敛时间长

伪随机序列方法：

matlab

```
% 注入已知的伪随机校准信号
calibration_signal = generate_pn_sequence();
total_input = user_signal + calibration_signal;

% ADC输出包含用户信号和校准信号
adc_output = adc_convert(total_input);

% 通过相关运算提取校准信息
correlation = correlate(adc_output, calibration_signal);
weight_error = extract_weight_error(correlation);

% 更新校准系数
cal_weights = update_weights(cal_weights, weight_error);
```

3. 工厂校准 (Factory Calibration)

特点：

- 生产时一次性校准
- 使用高精度测试设备
- 校准系数存储在非易失存储器中

测试设置：



校准效果评估

校准前后对比

matlab

% 校准前的传递函数

```
ideal_codes = 0:2^n_bits-1;
```

```
actual_voltages_uncal = ideal_codes * LSB_nominal;
```

% 校准后的传递函数

```
cal_weights = load_calibration_data();
```

```
actual_voltages_cal = ideal_codes * cal_weights;
```

% 计算INL改善

```
INL_before = calculate_INL(actual_voltages_uncal);
```

```
INL_after = calculate_INL(actual_voltages_cal);
```

```
improvement = max(abs(INL_before)) / max(abs(INL_after));
```

```
fprintf('INL improvement: %.1fx\n', improvement);
```

校准精度极限

校准本身也有误差：

$$\sigma^2_{\text{total}} = \sigma^2_{\text{ADC}} + \sigma^2_{\text{calibration}}$$

其中：

σ^2_{ADC} ：ADC本身的随机误差

$\sigma^2_{\text{calibration}}$ ：校准过程的误差

校准收益：当 $\sigma_{\text{calibration}} \ll \sigma_{\text{systematic}}$ 时才有意义

实际应用考虑

1. 存储需求

N位ADC需要存储N个权重值

每个权重用16-24位表示

总存储：N × 16 bits

例如：16位ADC需要 16×16 = 256 bits的校准存储

2. 计算复杂度

校准后的输出计算:

$$\text{Output} = \sum_{i=0}^{N-1} B_i \times W_i$$

需要N次乘法和N-1次加法

对于高速ADC，需要并行乘累加器

3. 校准更新策略

温度变化: 每10°C更新一次

时间漂移: 每月更新一次

电源变化: 监测电源，必要时触发校准

总结

这些概念构成了现代高性能ADC的技术基础:

- 采样保持**确保信号在转换期间稳定
- 量化噪声**决定了ADC的理论性能极限
- 余差放大**是Pipeline ADC实现高精度的关键
- 多级量化**平衡了速度、精度和复杂度
- 码字拼接**保证了多级结构的正确输出
- 权重校准**补偿了工艺误差，实现了设计精度

理解这些概念及其相互关系，对于ADC的设计、测试和应用都至关重要。